Part A:

Experiment 3: Disassemble the suspicious binary and analyse its code flow to determine what actions the malware is programmed to perform. Identify any suspicious API calls or functions it uses that could indicate malicious intent.

Tool: Ghidra

Static malware analysis involves examining an executable **without running it** to understand its behavior. Disassembly and decompilation reveal program logic, API usage, and control flow, which help determine whether the binary performs malicious actions such as persistence, file manipulation, or network communication.

PART A: Creating the Suspicious Binary (Kali Linux VM)

Step 1: Install MinGW-w64 (Kali Linux VM)

sudo apt update

sudo apt install mingw-w64

Step 2: Create Working Directory

mkdir ghidra_lab

cd ghidra_lab

Step 3: Create Suspicious Source code

```c
#include<windows.h>
#include<stdio.h>
int main()
{
HKEY hKey;
HANDLE hFile;
DWORD bytesWritten;
char data[]="System Initialized\n";

hFile=CreateFileA("C:\\Temp\\syslog.txt",GENERIC_WRITE,0,NULL,CREATE_ALWAYS,FILE_ATTRIBUTE_HIDDEN, NULL);

WriteFile(hFile,data,sizeof(data), &bytesWritten, NULL);
CloseHandle(hFile);

RegCreateKeyExA(
HKEY_CURRENT_USER, "Software\\Mirosoft\\Windows\\CurrentVersion\\Run",0, NULL, 0, KEY_WRITE, NULL, &hKey, NULL);

RegSetValueExA(hKey,"SystemUpdater", 0,REG_SZ, (BYTE*)"C:\\Temp\\Sample.exe",20);
RegCloseKey(hKey);
MessageBoxA(NULL,"Service Started","System", MB_OK);
return 0;
}
```

Step 3: Create Suspicious Source Code

nano sample.c

#include <windows.h>

#include <stdio.h>

int main() {

HKEY hKey;

HANDLE hFile;

DWORD bytesWritten;

char data[] = "System initialized\n";

// Create a hidden file (suspicious behavior)

hFile = CreateFileA("C:\\Temp\\syslog.txt", GENERIC_WRITE, 0,NULL,CREATE_ALWAYS, FILE_ATTRIBUTE_HIDDEN, NULL);

WriteFile(hFile, data, sizeof(data), &bytesWritten, NULL);

CloseHandle(hFile);

// Add registry persistence

```
    RegCreateKeyExA(HKEY_CURRENT_USER,    "Software\\Microsoft\\Windows\
\CurrentVersion\\Run", 0,NULL,0,KEY_WRITE,NULL,&hKey,        NULL);
    RegSetValueExA(hKey,"SystemUpdater",  0,REG_SZ,(BYTE*)"C:\\Temp\\sample.exe", 20);
    RegCloseKey(hKey);
    MessageBoxA(NULL, "Service Started", "System", MB_OK);
    return 0;
}
```

Save and exit.

Step 4: Compile Windows Executable in Kali Linux VM

x86_64-w64-mingw32-gcc sample.c -o sample.exe

Step 5: Verify Binary Type

file sample.exe

Expected output:

PE32+ executable (console) x86-64, for MS Windows

✔ Suspicious binary successfully created.

PART B: Preparing Ghidra (Kali Linux)

Then before installing Ghidra, make sure

Jdk is installed

sudo apt install -y openjdk-21-jdk

Step 7: Install Ghidra

sudo apt install -y ghidra

Step 7: Launch Ghidra

ghidra

Step 8: Create a New Project

1. **File → New Project**
2. Select **Non-Shared Project**
3. Project name: Malware_Ghidra_Analysis
4. Choose workspace directory

Step 9: Import the Suspicious Binary

1. **File → Import File**
2. Select sample.exe
3. Format: **PE**
4. Click **OK**

Step 10: Analyze the Binary

When prompted:

✔ Analyze
✔ Default analyzers
✔ PE analyzers
✔ Decompiler

Click **Analyze**

PART C: Initial Binary Inspection

1. Symbol Tree (Functions & Imports)

How to open

Menu bar:

Window → Symbol Tree

What you observe

This shows:

- **Functions**
- **Imported APIs**
- **Exported symbols**
- **Labels**

Expand:

Symbol Tree
 ├── Functions
 ├── Imports
 └── Labels

What to check for your sample.exe

Under **Imports**, you should see APIs like:

CreateFileA
WriteFile
CloseHandle
RegCreateKeyExA
RegSetValueExA
MessageBoxA

These APIs reveal the **capabilities of the executable**.

Example:

API	**Meaning**
CreateFileA	File creation
WriteFile	Writing data
RegCreateKeyExA	Registry key creation
RegSetValueExA	Registry persistence
MessageBoxA	Popup message

2. Defined Strings

How to open

Window → Defined Strings

What you observe

This panel shows **all text strings inside the executable**.

For your program you should see:

System Initialized
C:\Temp\syslog.txt
Software\Microsoft\Windows\CurrentVersion\Run
SystemUpdater
C:\Temp\Sample.exe
Service Started
System

Why it is important

Strings reveal:

- file paths
- registry keys
- messages
- commands
- URLs

Analysts often **start analysis here**.

3. Decompiler Window

How to open

Window → Decompiler

What you observe

This shows **C-like reconstructed code** from machine instructions.

Example you might see:

```
CreateFileA("C:\\Temp\\syslog.txt", ...);
WriteFile(...);
RegCreateKeyExA(...);
```

RegSetValueExA(...);
MessageBoxA(...);

This helps you understand the program **without reading assembly**.

4. Listing Window (Assembly View)

How to open

Window → Listing

What you observe

This shows the **actual assembly instructions** of the program.

Example:

PUSH offset "C:\\Temp\\syslog.txt"
CALL CreateFileA
CALL WriteFile
CALL RegCreateKeyExA

This is the **real machine-level program execution**.

Malware analysts verify behavior here.

RESULT

The suspicious binary was successfully disassembled using Ghidra. Analysis of its code flow and API calls revealed that it creates hidden files, establishes registry persistence, and executes automatically, indicating malicious intent.

CONCLUSION

Ghidra static analysis revealed that the analyzed binary performs unauthorized file system modification and registry persistence using Windows API calls. These behaviors are characteristic of malware designed to maintain persistence and operate stealthily on infected systems.